Time Complexity Analysis for USEI12

Kruskal's MST Algorithm

Method: `kruskalMST()`

Where:

- **V**: Number of vertices (Stations) in the graph
- **E**: Number of edges (TrackConnection) in the graph

---

1. Get vertices and initialize Temp MST: `O(V)`

- `railwayGraph.vertices()` : O(V)
- `mst.addVertex(v)` loop: O(V)

2. Get all edges: `O(E)`

- `railwayGraph.edges()` : O(E)

Looping through all the edges and using the `List.add()` method would give the same complexity, since we are looping through all edges anyway and `add()` method as a complexity of O(1) therefore being negligible.

3. Sort edges by cost: `O(E log E)`

- `edges.sort(Comparator.comparingDouble(...))` : O(E log E)

4. Main loop: `O(E × V)`

- Loop through E edges
- For each edge:
  - `Algorithms.DepthFirstSearch(mst, vOrig)` : O(V + E) in worst case
  - `validVertex()` : O(1)
  - `addVertex()` : O(1)
  - `contains()` : O(V) in the worst case
  - `addEdge()` : O(1)
- Total: O(E × V)

5. Preparing final MST

- Adding vertices: O(V)
- Iterating edges and filtering: O(E)
- Combined: O(V + E)

Overall Complexity:

- DFS dominates for dense graphs: **O(E × V)** So the overall complexity is O($E^2 \cdot V$) due to the DFS check inside the edge loop